



Copyright Infopro Digital Limited 2025. All rights reserved. You may share using our article tools. This article may be printed for the sole use of the Authorised User (named subscriber), as outlined in our terms and conditions. <https://www.infopro-insight.com/termsconditions/insight-subscriptions>

Research Paper

Deep self-consistent learning of local volatility

Zhe Wang,¹ Ameir Shaa,^{1,2} Nicolas Privault² and Claude Guet²

¹CNRS@CREATE, 08-01 Create Tower, 1 Create Way, Singapore 138602, Singapore; emails: zhe.wang@cnrsatcreate.sg, ameir.shaa@cnrsatcreate.sg

²School of Physical and Mathematical Sciences, Nanyang Technological University (NTU), 21 Nanyang Link, Singapore 637371, Singapore; emails: nprivault@ntu.edu.sg, cguet@ntu.edu.sg

(Received November 30, 2021; revised November 27, 2024; accepted March 18, 2025)

ABSTRACT

We present an algorithm for the calibration of local volatility from market option prices through deep self-consistent learning, by approximating both market option prices and local volatility using deep neural networks. Our method uses the initial–boundary-value problem of the underlying Dupire partial differential equation solved by the parameterized option prices to provide corrections to the parameterization in a self-consistent way. By exploiting the differentiability of neural networks, we can evaluate Dupire’s equation locally at each strike–maturity pair; while by exploiting their continuity, we sample strike–maturity pairs uniformly from a given domain, going beyond the discrete points where the options are quoted. Moreover, the absence of arbitrage opportunities is imposed by penalizing an associated loss function as a soft constraint. For comparison with existing approaches, the proposed method is tested on both synthetic and market option prices, and it shows an improved performance in terms of reduced interpolation and repricing errors, as well as the smoothness of the calibrated local volatility. An ablation study is performed, asserting the robustness and significance of the proposed method.

Keywords: local volatility; deep learning; option pricing; Dupire partial differential equation (PDE); physics-informed machine learning; self-consistent algorithm.

1 INTRODUCTION

An option is a financial contract that gives options holders the right but not the obligation to buy (a call option) or to sell (a put option) an asset (eg, stocks or commodity) for a predetermined price (strike price) on a predetermined date (maturity). To acquire today the right to buy or sell an asset in the future, a premium (namely, the option price) must be paid to the option writers. The standard approach to modeling option prices is based on the Black–Scholes formula, which assumes a constant local volatility surface and therefore applies only to situations where the stock prices process follows a geometric Brownian motion. However, the market price data of underlying assets typically invalidates the assumptions of standard Brownian motion; hence, a more sophisticated model than the Black–Scholes formula is required. Among alternatives, we consider an asset price process expressed in a local volatility model as

$$dS_t = rS_t dt + \sigma(S_t, t)S_t dB_t, \quad S_t|_{t=0} = S_0, \quad (1.1)$$

where S_t denotes the stock price at time $t \in \mathbb{R}_+$, and $(B_t)_{t \in \mathbb{R}_+}$ is a standard Brownian motion. For simplicity, we assume a zero dividend yield and a constant risk-free interest rate, r . Rather than taking a constant value as in the canonical Black–Scholes equation, in (1.1) the local volatility $\sigma(S_t, t)$ is a deterministic function of the underlying asset price S_t and of time t , satisfying the usual Lipschitz conditions. Without introducing additional sources of randomness, the local volatility model is the only complete and consistent model that allows hedging based on solely the underlying asset (see Bennett 2014, Appendix A1), and it is used for daily risk management in most investment bank production systems.

The practical implementation of such a stochastic volatility model in option pricing requires us to solve the challenging problem of calibrating the local volatility function to the market data of option prices. Given $\pi(K, T)$, a family of option prices with strike prices $K > 0$, maturities $T > 0$ and underlying asset price $S_0 = x$, the Dupire formula (Derman and Kani 1994; Dupire 1994),

$$\sigma(K, T) := \sqrt{\left(2\frac{\partial \pi}{\partial T}(K, T) + 2rK\frac{\partial \pi}{\partial K}(K, T)\right)\left(K^2\frac{\partial^2 \pi}{\partial K^2}(K, T)\right)^{-1}}, \quad (1.2)$$

provides a solution to this problem by constructing an estimator of the local volatility as a function of the strike price K and maturity T values; that is, (1.2) matches the stock's volatility $\sigma(S_t, t)$ when the underlying stock price S_t is at level K at time $T = t$.

The numerical estimation of local volatility using (1.2) involves evaluating partial derivatives, which is achieved using the classical finite-difference method. More

efficient methods have been introduced that use spline functions (see, for example, Achdou and Pironneau 2005, Chapter 8), where local volatility is first approximated as the sum of a piecewise affine function satisfying a boundary condition and a bi-cubic spline function. Then, the model parameters are determined by minimizing the discretized Dupire (1994) equation at selected collocation points via Tikhonov regularization. Finally, Tikhonov regularization is applied to the calibration of local volatility in a trinomial model of Crépey (2002).

As an alternative to calibrating local volatility using parameterized functions, neural networks are able to approximate any arbitrary nonlinear function with a finite set of parameters (Gorban and Wunsch 1998; Lin and Jegelka 2018; Winkler and Le 2017). By exploiting this property, Chataigner *et al* (2020) advocated parameterizing market option prices $\pi(K, T)$ using neural networks, which allows for the computation of the derivatives in (1.2) by automatic differentiation (Baydin *et al* 2018).

In addition, for Dupire's formula to be meaningful, we must ensure that the argument in the square root is positive and remains bounded in (1.2), which can be achieved under a no-arbitrage condition for the quoted option prices. Chataigner *et al* (2020) impose additional constraints in order to ensure the market option prices are well fitted, in which case an approximation of the derivatives of the option price is then substituted into (1.2) to calibrate the local volatility.

In this case, the quality of the reconstructed local volatility surface depends on the resolution and quality of the quoted market option prices at given strike–maturity pairs. Hence, instead of using the market option prices, Chataigner *et al* (2021) subsequently proposed the use of the implied volatility surface for the calibration of local volatility, leading to improved performance. It is important to note, however, that when fitting option prices or implied volatilities using neural networks, we assume the data is noise free. Except for those that violate the arbitrage-free conditions, the proposed methods of Chataigner *et al* (2020, 2021) are unable to filter out noise in the data.

In this paper, we follow the approach of Wang and Guet (2022b) and calibrate the local volatility surface from the observed market option prices in a self-consistent manner. Going beyond merely fitting option prices using neural networks, we rely on recent studies on physics-informed machine learning (see, for example, Karniadakis *et al* 2021; Rackauckas *et al* 2020; Raissi *et al* 2019; Wang and Guet 2022b), which have shown that the inclusion of the residual of an unknown, underlying governing equation as a regularizer can not only calibrate the unknown terms in the governing equations but also correct the data by filtering out noise that is not defined by the equation.

To do this, we approximate both the market option prices and the squared local volatility using deep neural networks. By requiring $\pi(K, T)$ to be a solution to

the Dupire equation subject to given initial and boundary conditions (to be discussed in Section 2.1), we regularize $\pi(K, T)$ and determine the unknown $\sigma(K, T)$ self-consistently. Since the resulting $\pi(K, T)$ and $\sigma(K, T)$ are continuous and differentiable functions of K and T , we can evaluate Dupire's equation locally on a uniformly sampled (K, T) from the support of Dupire's equation. Unlike previous approaches, where constraints were imposed on fixed-strike–maturity pairs, successive resamplings enable us to regularize the entire support. In addition, the positiveness of $\sigma(K, T)$ is ensured by a properly selected output activation function of the neural network; hence, it is well defined everywhere. Finally, we mitigate the risk of arbitrage opportunities by penalizing a loss as a soft constraint. In this way, local volatility is estimated, by construction, as a smooth surface instead of by applying automatic differentiation to neural option prices. In addition, the noise in market data is filtered out via the constraint imposed by Dupire's equation. A TENSORFLOW implementation of our algorithm is available from the “Local volatility modeling with neural networks” GitHub repository.¹

The rest of the paper is structured as follows. In Section 2 we provide background information on Dupire's equation and arbitrage-free conditions, followed by a comment on market data. After the rescaling and reparameterization of market option prices in Section 3.1 and a discussion of the neural ansatz and loss functions in Section 3.2, our deep self-consistent algorithm is summarized and discussed in Section 3.3. The proposed method is first tested on synthetic data sets in Section 4.1, and then applied to market option prices in Section 4.2, where, as an ablation study, we compare our results with those obtained without including the residual of Dupire's equation as a regularizer. Finally, the advantages and limitations of our method are discussed in Section 5, where we state our conclusions.

2 BACKGROUND AND OBJECTIVES

2.1 Dupire's equation

Denoting by $\pi^c(K, T)$ and $\pi^p(K, T)$ the prices for European call and put options

$$\pi^c(K, T) = e^{-rT} \mathbb{E}[(S_T - K)^+], \quad \pi^p(K, T) = e^{-rT} \mathbb{E}[(K - S_T)^+], \quad (2.1)$$

the relation between the market option price and the local volatility is established explicitly by Dupire's equation (Dupire 1994),

$$\frac{\partial \pi}{\partial T}(K, T) + rK \frac{\partial \pi}{\partial K}(K, T) - \frac{1}{2} K^2 \sigma^2(K, T) \frac{\partial^2 \pi}{\partial K^2}(K, T) = 0, \quad (2.2)$$

¹ URL: <https://github.com/ameirtheshaa/LocalVolatility>.

subject to the initial conditions

$$\pi^c(K, 0) = (S_0 - K)^+, \quad \pi^p(K, 0) = (K - S_0)^+,$$

and to the boundary conditions

$$\pi^c(\infty, T) = 0, \quad \pi^p(0, T) = 0,$$

for call and put options, respectively. We refer the reader to Chapter 1 of Itkin (2020) or Proposition 9.2 of Privault (2022) for derivations of Dupire's equation, and to Chapter 8 of Achdou and Pironneau (2005) for a discussion of initial and boundary conditions. From the definitions of European options prices in (2.1), we note that these are nonnegative, with the call option price always being less than the value of the underlying asset, and the price of the put options always being less than the present value of the strike price, leading to the following lower and upper bounds:

$$0 \leq \pi^c(K, T) \leq S_0, \quad 0 \leq \pi^p(K, T) \leq K. \quad (2.3)$$

2.2 Arbitrage-free surface

Static arbitrage is a trading strategy that has zero initial cost, and subsequently a constant nonnegative value, representing a risk-free profitable investment. Under the assumption that economic agents are rational, arbitrage opportunities (if they ever exist) will be instantaneously exploited until the market is arbitrage-free. Therefore, as a prerequisite, the absence of arbitrage opportunities is a fundamental principle underpinning the modern theory of financial asset pricing. An option price surface is free of static arbitrage if and only if

- (i) each strike slice is free of calendar spread arbitrage; and
- (ii) each maturity slice is free of butterfly arbitrage (Ackerer *et al* 2020; Chataigner *et al* 2020, 2021).

Next, we consider both calendar spread and butterfly arbitrage, which can be created with put options as well as call options (see Hull (2003, Section 9.2) for details). Calendar spread arbitrage is created by buying a long-maturity put option and selling a short-maturity put option, and the absence of arbitrage can be characterized in the continuous limit by the condition

$$\frac{\partial \pi}{\partial T}(K, T) \geq 0.$$

On the other hand, butterfly arbitrage can be created by buying two options at prices K_1 and K_3 , with $K_3 > K_1$, and by selling two options with a strike price $K_2 = (K_1 + K_3)/2$. In the continuous limit, the absence of butterfly arbitrage is formulated as

$$\frac{\partial^2 \pi}{\partial K^2}(K, T) \geq 0.$$

2.3 Problem formulation

Market option prices are observed on N discrete pairs of strike prices and maturities, leading to the triplets (π_i, K_i, T_i) , $i = 1, \dots, N$. In practice, to secure a transaction, two option prices – a bid price and an ask price – are quoted in the market, and there is no guarantee that the midprice π_i between the bid and ask prices is arbitrage-free (Ackerer *et al* 2020). Moreover, the observed bid and ask prices may not be updated in a timely manner and hence may not be actionable, leading to additive noise in the market data. Lastly, market option prices are not evenly quoted over a plane spanned by K and T : the market data is usually dense close to the money but sparse away from the money. Canonical methods often lack the ability to agilely interpolate option prices and calibrate the associated local volatility without quotes. Therefore, validation of and correction to the market data and data interpolation are required.

Consequently, the main challenge in calibrating the local volatility is twofold. First, given limited observations of unevenly distributed option prices quoted on discrete pairs of strikes and maturities, the calibrated local volatility should span a continuous domain. Second, the corresponding option prices should preclude any arbitrage opportunities. Our objective is therefore to construct from observed market option prices a self-consistent approximation for both $\sigma(K, T)$ and $\pi(K, T)$, in the sense that the calibrated local volatility will generate an arbitrage-free option price surface that is in line with the market data up to some physics-informed corrections.

3 METHODOLOGY

3.1 Rescaling Dupire's equation

In this section, a change of variable and a rescaling are applied to Dupire's equation (2.2) to ensure that the terms in the scaled equation, (3.2), are of order unity, so that no term dominates the others during the minimization. To do this, we use the change of variables

$$k = \frac{e^{-rT} K}{K_{\max}}, \quad t = \frac{T}{T_{\max}}, \quad (3.1)$$

by picking sufficiently large $K_{\max} = \max(K)$ and $T_{\max} = \max(T)$. Letting

$$\eta(k, t) := \frac{1}{2} T_{\max} \sigma^2(K, T)$$

denote the squared local volatility, the Dupire equation, (2.2), can be rewritten as

$$f_{\text{dup}}(k, t) := \frac{\partial \pi}{\partial t}(k, t) - \eta(k, t) k^2 \frac{\partial^2 \pi}{\partial k^2}(k, t) = 0, \quad (3.2)$$

subject to the rescaled initial and boundary conditions

$$\pi^c(k, 0) = (S_0 - K_{\max} k)^+, \quad \pi^c(\infty, t) = 0, \quad (3.3)$$

for call option prices, and

$$\pi^P(k, 0) = (K_{\max}k - S_0)^+, \quad \pi^P(0, t) = 0, \quad (3.4)$$

for put option prices, as well as the inequality constraints (2.3). We note the conditions

$$\frac{\partial \pi}{\partial T}(K, T) \propto \frac{\partial \pi}{\partial t}(k, t) - rT_{\max}k \frac{\partial \pi}{\partial k}(k, t) \geq 0, \quad (3.5)$$

and

$$\frac{\partial^2 \pi}{\partial K^2}(K, T) \propto \frac{\partial^2 \pi}{\partial k^2}(k, t) \geq 0, \quad (3.6)$$

for the absence of strike and calendar arbitrage opportunities, respectively (see, for example, Bergomi 2016, Section 2.2.2).

LEMMA 3.1 *Conditions (3.5) and (3.6) for the absence of strike and calendar arbitrage can be simultaneously enforced under the following single inequality:*

$$f_{\text{arb}}(k, t) := \frac{\partial \pi}{\partial t}(k, t) - rT_{\max}k \left(\frac{\partial \pi}{\partial k}(k, t) \right)^+ \geq 0. \quad (3.7)$$

PROOF We consider two cases.

CASE 1 ($\partial \pi / \partial k \geq 0$) In this case, (3.7) is clearly equivalent to (3.5). In addition, if (3.5) holds, then (3.2) shows that (3.6) is satisfied, as

$$\eta(k, t)k^2 \frac{\partial^2 \pi}{\partial k^2}(k, t) = \frac{\partial \pi}{\partial t}(k, t) \geq rT_{\max}k \frac{\partial \pi}{\partial k}(k, t) \geq 0.$$

CASE 2 ($\partial \pi / \partial k < 0$) In this case, (3.7) becomes

$$\frac{\partial \pi}{\partial t}(k, t) \geq 0, \quad (3.8)$$

which by (3.2) is equivalent to (3.6). In addition, if (3.8) holds, then (3.5) is satisfied. $\square$

3.2 Model and loss function

Market option prices are observed at discrete strike–maturity pairs, whereas it is favorable to have an option price and a local volatility that span continuous surfaces over the extended regimes of strike price and maturity. To obtain a continuous limit, we propose to model both the option price and the local volatility using neural networks. The associated neural network ansatz and the corresponding loss functions are detailed below.

3.2.1 Neural network ansatz

As option prices may vary dramatically against strike prices, we model the option price surface as an exponential function where the exponent is approximated using a neural network, whose model parameters θ are learned from the market data. We consider the neural ansätze

$$\pi_{\theta}^c(k, t) = S_0(1 - e^{-(1-k)\mathcal{N}_c(k, t; \theta_c)}) \quad (3.9)$$

and

$$\pi_{\theta}^p(k, t) = K_{\max} e^{rT_{\max}t} k(1 - e^{-k\mathcal{N}_p(k, t; \theta_p)}) \quad (3.10)$$

for call and put option prices, respectively. At the practical infinity $k = 1$ and at $k = 0$, the boundary conditions for call options, (3.3), and put options, (3.4), are satisfied, by construction. Here, a softplus function, $\text{softplus}(x) := \log(1 + e^x)$, is selected to be the activation function at the output layer of the neural networks, so that the inequality constraints (2.3) are always satisfied.

Unlike for option prices, the variation in local volatility is relatively small; hence, we model it using the following neural network:

$$\eta_{\theta}(k, t) = \mathcal{N}_{\eta}(k, t; \theta_{\eta}), \quad \mathcal{N}_{\eta}: \mathbb{R}_+ \times \mathbb{R}_+ \rightarrow \mathbb{R}_+, \quad (3.11)$$

with a softplus activation function at the output layer, so that $\eta_{\theta}(k, t)$ is nonnegative by construction. By modeling the squared local volatility using a neural network we also ensure the smoothness of the local volatility, thus mitigating numerical instabilities.

Throughout the paper, we use the same neural network architecture for $\mathcal{N}_c$, $\mathcal{N}_p$ and $\mathcal{N}_{\eta}$. The network consists of three residual blocks (He *et al* 2016), where the residual connection is used around two sublayers formed of 64 neurons per layer and regularized with batch normalization (Ioffe and Szegedy 2015). The model parameters of the neural networks θ_c (or θ_p) and θ_{η} are determined by minimizing properly designed loss functions (to be discussed in Section 3.2.2). Since evaluating loss functions associated with the arbitrage-free conditions and with the Dupire equations requires computing successive derivatives, the tanh activation function is selected for the hidden network layers.

3.2.2 Loss functions

Given N market data triplets (π_i, K_i, T_i) , $i = 1, \dots, N$, we transform K_i and T_i using (3.1), leading to (π_i, k_i, t_i) , and we consider the following loss functions.

- (i) The data loss arising from fitting market option prices using neural networks is defined as the squared error,

$$L_{\text{fit}} = \frac{1}{N} \sum_{i=1}^N w(\pi_i) (\pi_{\theta}(k_i, t_i) - \pi_i)^2,$$

with the weight function

$$w(\pi_i) := 1 + \frac{Ng(\pi_i)}{\sum_{j=1}^N g(\pi_j)},$$

where

$$g(x_i) := \frac{1}{10} \mathbf{1}_{[0,0.1)}(x_i) + x_i \mathbf{1}_{[0.1,10]}(x_i) + 10 \mathbf{1}_{[10,\infty)}(x_i),$$

and

$$x_i = \text{stop_gradient}(\tilde{x}_i), \quad \text{with } \tilde{x}_i = \frac{1}{N\pi_i^2} \sum_{j=1}^N \pi_j^2, \quad (3.12)$$

so that squared and relative squared errors are minimized. Here, indicator functions are used to mitigate the numerical instabilities associated with $\pi_i = 0$ while leveraging contributions from large values of π_i . In addition, in (3.12) the operator `stop_gradient(·)` prevents the input variables of the indicator functions from being taken into account when computing gradients; hence, the weight function $w(\cdot)$ is treated as a scalar quantity during backpropagation. As the weight function w is designed to balance the losses evaluated at each collation point, it will be included in all loss functions to be defined in the following.

- (ii) As the initial conditions in (3.3) and (3.4) are piecewise linear and not differentiable, instead of imposing initial conditions as hard constraints in the neural ansatz, we choose to minimize the loss function

$$L_{\text{ini}} = \frac{1}{M_1} \sum_{j=1}^{M_1} w(\pi(k_j, 0)) (\pi_{\theta}(k_j, 0) - \pi(k_j, 0))^2,$$

where the reference function $\pi(k_j, 0)$ is defined in (3.3) for call options and in (3.4) for put options. To evaluate L_{ini} , M_1 synthetic collocation points are uniformly sampled from the interval $k_j \in [0, 1]$ at each iteration during the minimization.

- (iii) To ensure that the parameterized option price surface is arbitrage-free, at the M_2 synthetic collocation points (k_j, t_j) uniformly sampled from $\Omega := [0, 1] \times$

$[0, 1]$ we substitute $\pi_\theta(k_j, t_j)$ into the function $f_{\text{arb}}(k, t)$ defined in (3.7), and we penalize the locations where $f_{\text{arb}}(k_j, t_j)$ is negative, which results in the loss function

$$L_{\text{arb}} = \frac{1}{M_2} \sum_{j=1}^{M_2} w \left(\frac{\partial \pi_\theta}{\partial t}(k_j, t_j) \right) ((-f_{\text{arb}}(k_j, t_j))^+)^2$$

associated with the arbitrage-free conditions.

- (iv) Under the assumption that the price process of the option's underlying assets matches the local volatility model (1.1), $\pi_\theta(k, t)$ must be a solution to the rescaled Dupire equation, (3.2), up to some error terms. The self-consistency between the parameterized option price and the calibrated local volatility is therefore established by minimizing the residual $f_{\text{dup}}(k, t)$ defined in (3.2), leading to

$$L_{\text{dup}} = \frac{1}{M_2} \sum_{j=1}^{M_2} w \left(\frac{\partial \pi_\theta}{\partial t}(k_j, t_j) \right) (f_{\text{dup}}(k_j, t_j))^2,$$

where, similar to L_{arb} , the loss function arising from the Dupire equation is evaluated from the ensemble of uniformly sampled M_2 collocation points.

In addition, the initial condition provides reference option prices on collocation points sampled on the line $T = 0$ that are equivalent to the quoted market option prices. Successive resamplings then cover the entire support domain Ω , ensuring that both the arbitrage-free condition and the Dupire equation are satisfied everywhere. This not only avoids the need to design an adaptive mesh for Dupire's equation, as is done by Achdou and Pironneau (2005), but also mitigates any overfitting due to the uneven distribution of scarce data. Computing loss functions L_{arb} and L_{dup} involves evaluating differential operators, which is conveniently achieved by automatic differentiation (Baydin *et al* 2018).

Finally, the joint minimization of

$$L_\pi = L_{\text{fit}} + \lambda_{\text{ini}} L_{\text{ini}} + \lambda_{\text{arb}} L_{\text{arb}} + \lambda_{\text{dup}} L_{\text{dup}}, \quad \lambda_{\text{ini}}, \lambda_{\text{arb}}, \lambda_{\text{dup}} \in \mathbb{R}_+, \quad (3.13)$$

with respect to θ_c (or θ_p) provides a correction to the observed market data by filtering out noise that may lead to arbitrage opportunities or violate the price dynamics underpinned by the scaled Dupire equation. Without loss of generality, we take $\lambda_{\text{ini}} = \lambda_{\text{arb}} = 1$, reducing the number of hyperparameters to one: $\lambda_{\text{dup}} = \lambda \in \mathbb{R}_+$.

On the other hand, the minimization of L_{dup} with respect to θ_η ensures consistency between the local volatility function estimate and the parameterized option price. In this sense, the proposed method is self-consistent. To assess the robustness of the proposed self-consistent method, various values of λ are considered in Section 4 as an ablation study.

3.3 The algorithm

Based on the deep self-consistent learning method discussed above, Algorithm 3.2 outlines a computational scheme for the calibration of local volatility. At variance with the usual approaches whereby the derivatives of the option price are first approximated to calibrate the local volatility using Dupire's formula, (1.2), this algorithm approximates both the option price and the local volatility using neural networks.

ALGORITHM 3.2 (Deep self-consistent learning for option prices and local volatility)

Input: Scaled market call (or put) option prices and transformed strike–maturity pairs (π_i, k_i, t_i) , $i = 1, \dots, N$.

Guess initial parameters θ_c (or θ_p) and θ_η .

- 1: **while** not converged **do**
- 2: Uniformly draw M_1 points $k \in [0, 1]$ and M_2 scaled strike–maturity pairs $(k, t) \in \Omega$.
- 3: Compute four loss functions, L_{fit} , L_{ini} , L_{arb} , L_{dup} , and their weighted sum L_π .
- 4: Optimize θ_c (or θ_p) using gradients $\partial_{\theta_\pi} L_\pi$.
- 5: Optimize θ_η using gradients $\partial_{\theta_\eta} L_{\text{dup}}$.
- 6: **end while**

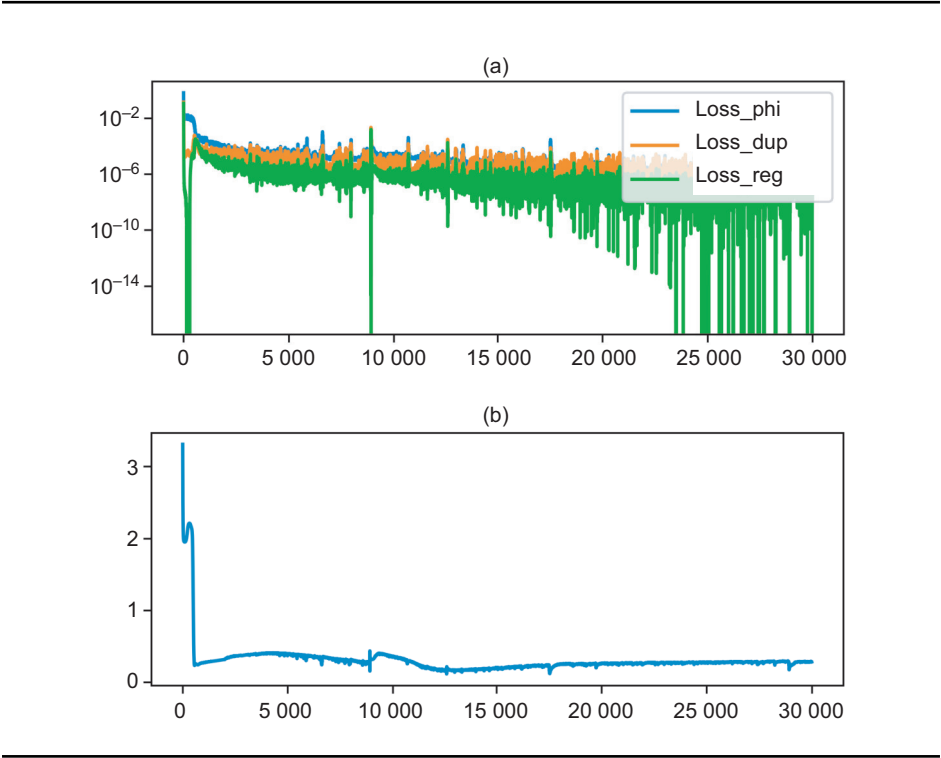
Output: Optimized parameters θ_c (or θ_p) and θ_η .

Hyperparameters used in this paper: $M_1 = 128$, $M_2 = 128^2$, $\lambda_{\text{ini}} = \lambda_{\text{arb}} = 1$ and $\lambda_{\text{dup}} = \lambda$.

The training of neural networks is a nonconvex optimization problem that does not guarantee convergence toward the global minimum; hence, regularization is required and provided via the residual of the scaled Dupire equation in the term L_{dup} . The joint minimization of L_{fit} arising from the deviation of the parameterized option price to the market data, L_{ini} due to the initial conditions, L_{arb} linked with the arbitrage-free conditions and L_{dup} seeks a self-consistent pair of approximations for the option price and for the underlying local volatility, and to exclude local minimums that violate these constraints.

Moreover, in practice, strike–maturity pairs with quoted option prices are usually unevenly distributed (Ackerer *et al* 2020; Chataigner *et al* 2021); hence, the direct minimization of loss functions evaluated on the quoted strike–maturity pairs may yield large local volatility calibration errors at locations where measurements are scarce. On the other hand, being continuous functions, neural networks allow for the mesh-free discretization of Dupire's equation by uniformly sampling collocation points from Ω at each iteration, thus improving on the calibration from

FIGURE 1 (a) Violations of the nonarbitrage condition and (b) relative local volatility error (σ).



scarce and unevenly distributed data. Lastly, since both the option price and squared local volatility are approximated using neural networks, the smoothness of the option prices and local volatility surface is guaranteed, while positiveness is ensured by a proper choice of output activation functions. As a result, the numerical instabilities associated with the calibration of local volatility using the Dupire formula, (1.2), are less likely to occur.

4 CASE STUDIES

In this section, we test our self-consistent method, first on synthetic option prices (in Section 4.1) and then on real market data (in Section 4.2). We start the training with an initial learning rate of 10^{-3} , which is divided by a decaying factor of 1.1 every 2000 iterations. On a workstation equipped with an Nvidia RTX 3090 graphics card, each iteration takes less than 0.01 seconds. The total number of iterations is capped at 30 000 such that the overall computation time is compatible with that of Chataigner

et al (2020, 2021), enabling a fair comparison. Figure 1 presents a typical graph of loss functions L_{ini} , L_{dup} and L_{arb} observed in the following experiments as the number of iterations increases.

Figure 2 presents a heatmap of the function f_{arb} , showing the no-arbitrage violations at various collocation points.

4.1 Synthetic data for European call options

Synthetic option prices are generated as Monte Carlo estimates of (2.1) for European call options at given strike–maturity pairs (K_i, T_i) . Here, the asset price paths S_t are obtained by simulating the local volatility model (1.1) with $r = 0.04$ and

$$\sigma(x, t) = 0.3 + ye^{-y}, \quad y = (t + 0.1)\sqrt{x + 0.1}, \quad (4.1)$$

making it possible to assess the validity of the calibrated local volatility by comparison with the closed-form expression of $\sigma(x, t)$. The price trajectories are obtained by simulating (1.1) 10^6 times from a single initial condition $S_0 = 1000$ for the period $t = [0, 1.5]$, with a time step $dt = 0.01$.

4.1.1 Large data set

To begin with, we price European call options on a mesh grid consisting of 10 evenly spaced points in $[0.3, 1.5]$ for period T and 20 evenly spaced points K in the interval $[500, 3000]$, respectively. That is, the data set consists of 10×20 option prices quoted at the corresponding strike–maturity pairs, leading to the triplet (π_i^c, K_i, T_i) , $i = 1, \dots, 200$. The exact local volatility surface, the price trajectories simulated by numerically integrating the local volatility model, and the synthetic call option price surface are visualized in Figure 3.

After θ_c and θ_η are determined, the parameterized option price and calibrated local volatility are recovered from (3.9) and (3.11). Thereafter, we substitute the calibrated neural local volatility into (1.1) to create a neural local volatility model, whose solution, in turn, gives the synthetic stock prices, from which the option price can then be recovered using Monte Carlo estimates of (2.1), completing a control loop for model assessment. The calibrated local volatility, the price trajectory simulated by numerically integrating the neural local volatility model with a presampled sequence of Brownian motion (B_t) , and the relative errors of calibrated local volatility with respect to the exact expression (4.1) are shown in Figure 4.

For $\lambda_{\text{dup}} = 0$, the parameterized option price is not regularized by the a priori physics information encoded in Dupire's equation, reducing the self-consistent calibration of local volatility to a one-way approach. In the cases in both parts (a) and (b) the parameterized option prices are regularized by the arbitrage-free conditions as in the works by Ackerer *et al* (2020) and Chataigner *et al* (2020, 2021). On the other

FIGURE 2 Heatmaps of (a) nonarbitrage condition violations and (b) linear interpolation.

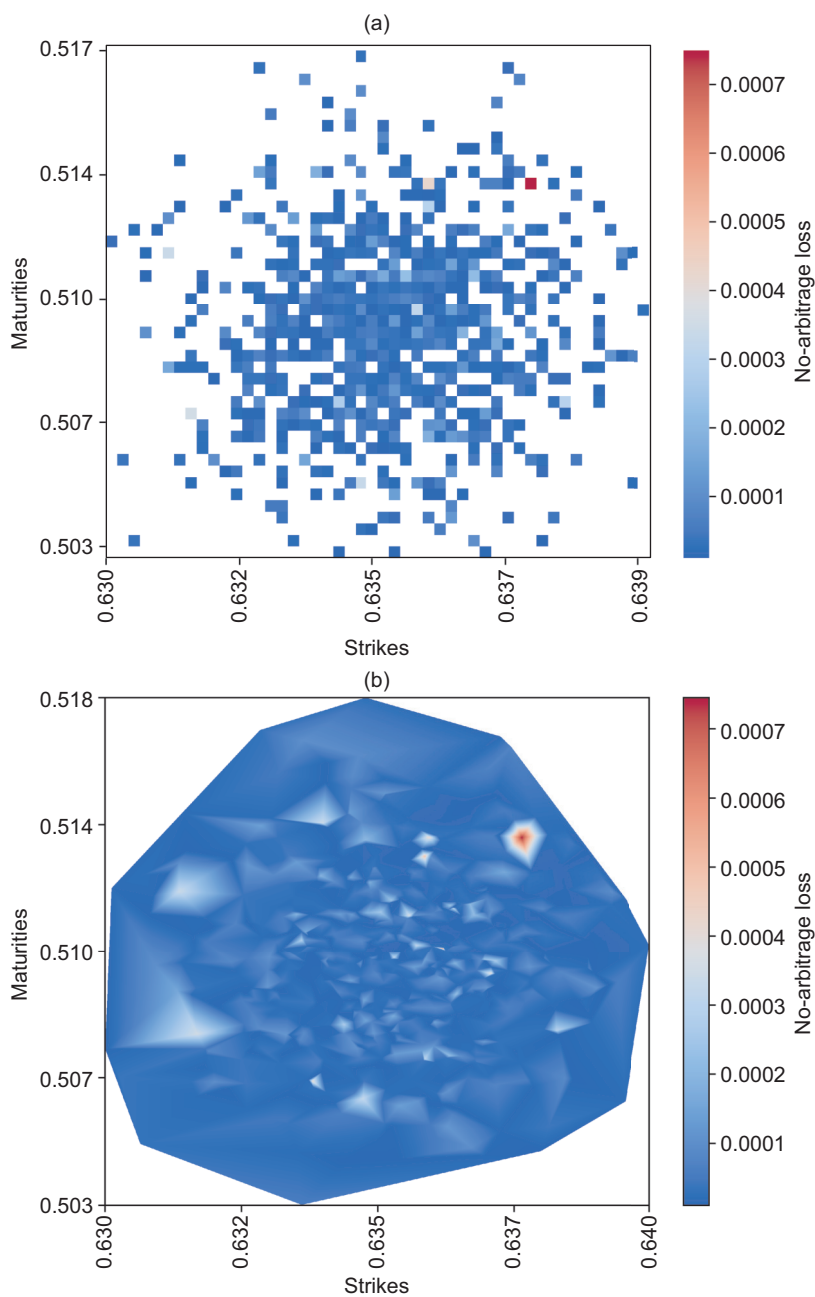
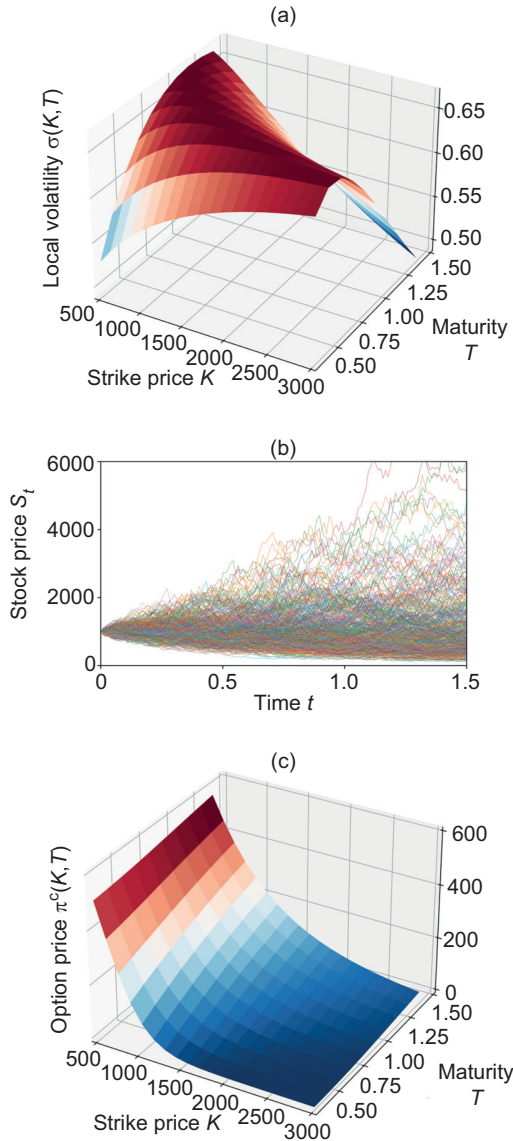
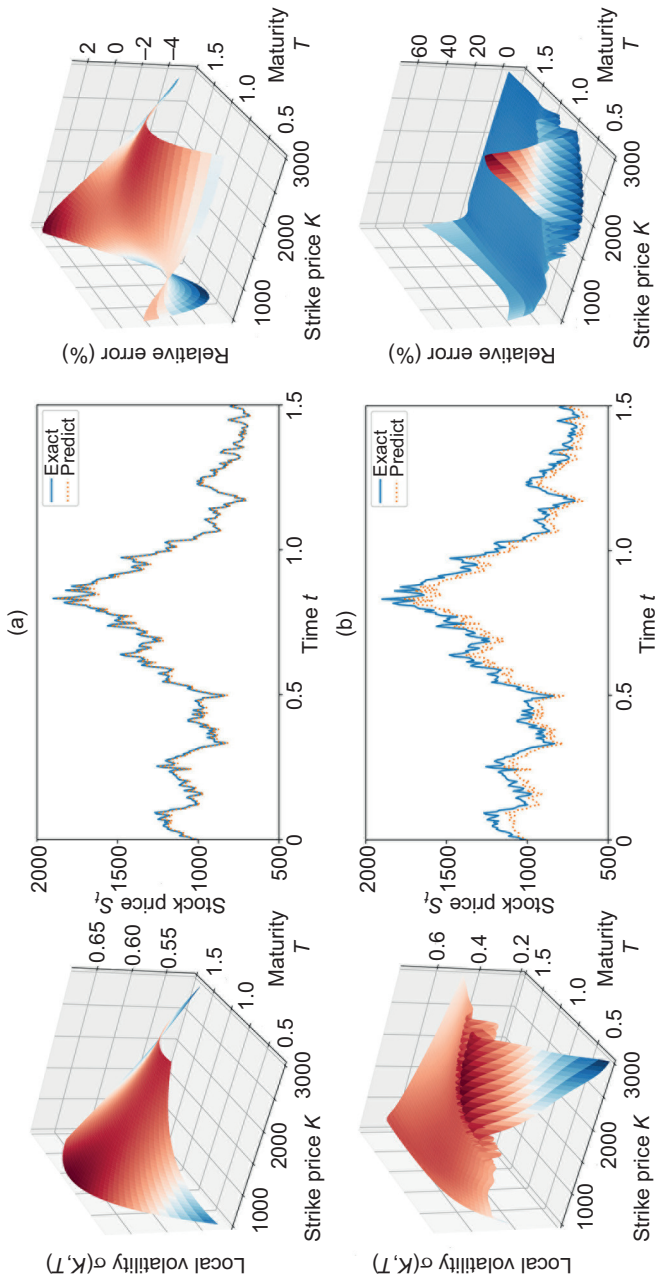


FIGURE 3 Exact local volatility surface, simulated price trajectories and generated call option price.



(a) Exact local volatility surface (see (4.1)). (b) Price trajectories simulated by solving the local volatility model (see (1.1)). (c) Call option price generated using Monte Carlo simulation of (2.1) with 10^6 sampled price trajectories. The local volatility and the option price surfaces are visualized on an evenly spaced 10×20 grid and, for better visualization, only the first 1024 price trajectories are shown in part (b).

FIGURE 4 Visualization, for a large data set, of calibrated neural local volatility, simulated price trajectory with a presampled Brownian motion (B_t) and the relative error of the calibrated local volatility with respect to (4.1) with and without L_{dup} as regularizer.



The figure shows, from left to right, the calibrated neural local volatility, simulated price trajectory with a presampled Brownian motion (B_t) and the relative error of the calibrated local volatility with respect to the exact expression (4.1), where in part (a) $\lambda_{\text{dup}} = 1$, with L_{dup} included as regularizer, and in part (b) $\lambda_{\text{dup}} = 0$, without L_{dup} as regularizer. The models shown are trained on a data set of 10×20 option prices, while the visualization and relative errors are computed on a different test grid with 256×256 linearly spaced collocation points.

hand, exploiting self-consistency by the inclusion of L_{dup} as a regularizer improves significantly the calibrated local volatility surface.

4.1.2 Small data set

To explore the small data limit of the proposed method, we present in Figure 5 the calibrated local volatility surface from a scarce data set consisting of 3×6 option prices quoted on a linearly spaced grid $(K, T) \in [500, 3000] \times [0.3, 1.5]$.

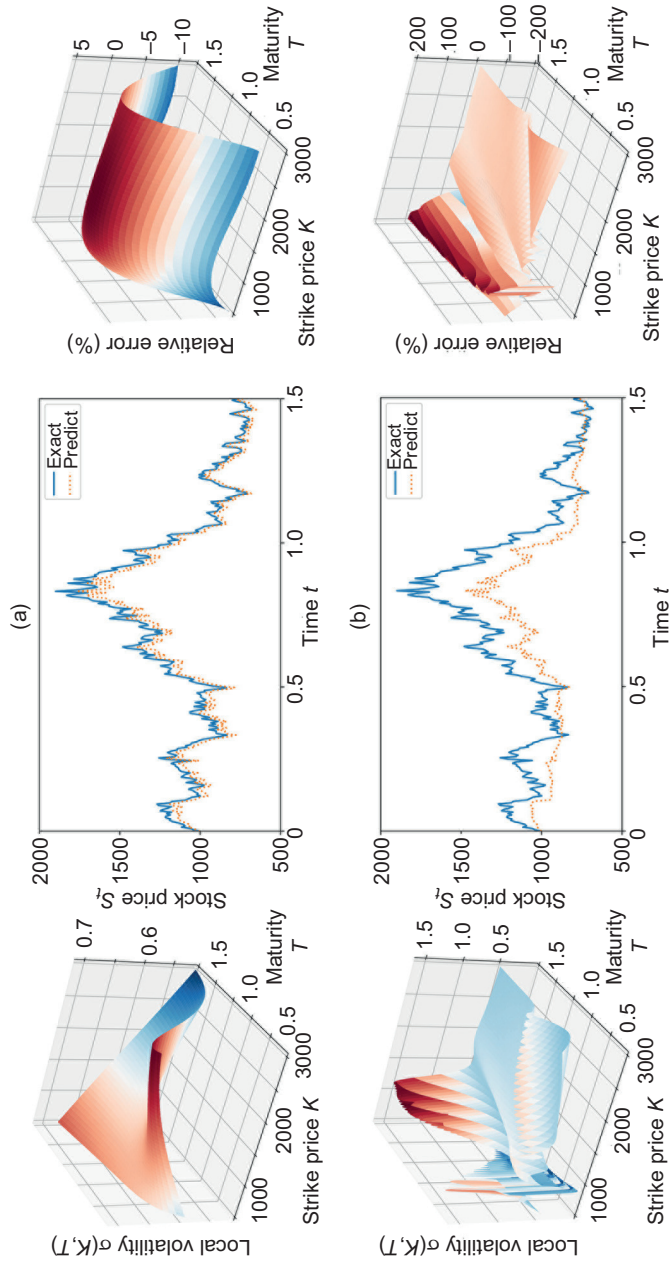
We can observe that the use of self-consistency effectively compensates for the lack of data, validating the proposed method in the small data limit. Indeed, neural networks are essentially nonlinear functions. Given discrete pairs of inputs and outputs, such a mapping function is not uniquely determined. To ensure that the obtained neural networks can be generalized, we need to go through the entire domain of definition of neural networks, calling for big data. Alternatively, incorporating a priori physics information, completely or partially known, into the learning scheme can regularize neural networks at places where there is no data, allowing us to benefit from recent advances in deep learning without a large data set.

4.1.3 Sensitivity with respect to λ_{dup}

To investigate the sensitivity of the proposed method with respect to λ_{dup} , we consider a sequence of values in the range $\lambda_{\text{dup}} = [0, 4]$. We quantify the accuracy of our self-consistent method by the root mean square error (RMSE) for the parameterized option price and the calibrated local volatility, which is less relevant due to the nonuniqueness of the solution to such calibration problems. Here, the RMSEs are computed on a different testing grid, independent of the training data set, consisting of 256×256 linearly spaced collocation points in $[K, T] \in [500, 3000] \times [0.3, 1.5]$. We perform three independent runs for each case, and the averaged results are summarized in Table 1.

We can observe that, when we increase λ_{dup} from zero, the RMSEs for the calibrated volatility and the repriced option price first decrease then increase, which is indicative of the existence of an optimal λ_{dup} . Its value seems to be dependent on the size of the data set, but its determination is beyond the scope of this paper and thus left for future work. The decrease in RMSE values for any reasonably chosen $\lambda_{\text{dup}} \neq 0$ support the inclusion of L_{dup} for regularizing the parameterized option price. With randomly initialized θ_η , the corresponding Dupire equation essentially provides the wrong a priori condition for the measured data. During the training, a joint minimization of L_π with respect to θ_c and of L_{dup} with respect to θ_η yields a self-consistent pair of parameterized option prices that provides an optimized description of the measured data, the initial and boundary conditions, the arbitrage-free conditions and the underlying Dupire equation, as well as for a cal-

FIGURE 5 Visualization, for a small data set, of calibrated neural local volatility, simulated price trajectory with a presampled Brownian motion (B_t) and the relative error of the calibrated local volatility with respect to (4.1) with and without L_{dup} as regularizer.



The models shown are trained on a scarce data set consisting of 3×6 option prices quoted on a linearly spaced grid $(K, T) \in [500, 3000] \times [0.3, 1.5]$

TABLE 1 Test of the proposed self-consistent method on a synthetic data set.

Data set	RMSEs	λ_{dup}				
		0.0	0.5	1.0	1.5	2.0
3×6	Option price	10.53	2.27	1.95	1.62	1.76
	Local volatility	0.25	0.03	0.03	0.02	0.03
	Reprice error	64.54	1.84	1.88	1.81	1.71
10×20	Option price	1.07	1.76	0.94	1.10	1.22
	Local volatility	0.04	0.01	0.01	0.01	0.02
	Reprice error	5.99	1.23	1.16	1.08	1.34

ibrated local volatility that matches, upon a physics-informed correction, the price dynamics of the observed data. Assigning too small a weight leads to unregularized parameterization, whereas a large λ_{dup} slows down the minimization, resulting in a larger calibration error after a fixed number of iterations.

4.2 Application to market data

In the case of market option prices, since the exact local volatility is not available, the assessment is performed by computing the reprice error. More specifically, we replace $\sigma(x, t)$ in (1.1) by the calibrated one,

$$\sigma_{\theta}(x, t) = \sqrt{\frac{2\eta_{\theta}(x, t)}{T_{\max}}}, \quad (4.2)$$

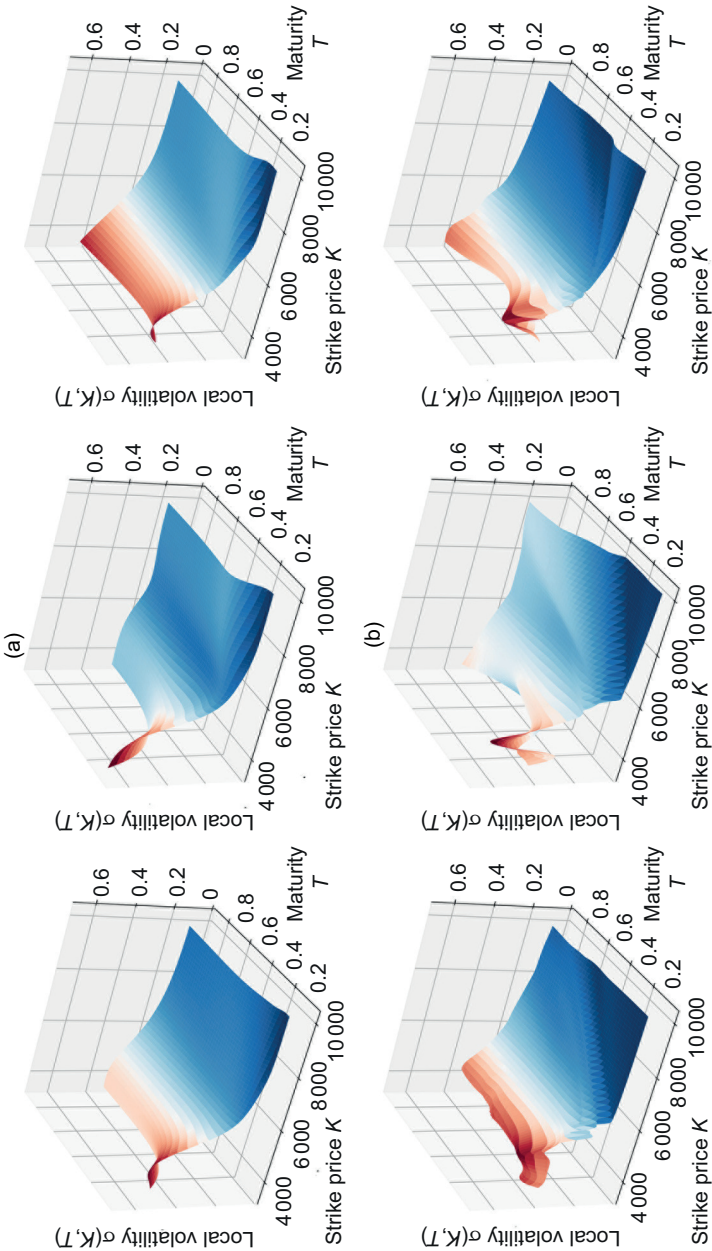
and we generate synthetic asset price paths by numerically integrating the neural local volatility model. The option is then repriced at given strike–maturity pairs using Monte Carlo estimates of (2.1) and, eventually, compared with the market call (or put) option prices.

4.2.1 European call options on the DAX index

To perform a comparison with the literature (see, for example, Chataigner *et al* 2020; Crépey 2002), we take the daily data set European call options listed on the Deutscher Aktienindex (DAX) index on August 7–9, 2001. The spot prices for the underlying assets are $S_0 = 5752.51, 5614.51, 5512.28$, respectively, with 217, 217 and 218 call options quoted at different strike–maturity pairs and $r = 0.04$. The local volatility surfaces calibrated with and without the inclusion of L_{dup} as a regularizer are shown in Figure 6.

It is observed that, compared with the case of $\lambda_{\text{dup}} = 0$, the inclusion of L_{dup} provides a greater stability of the local volatility surfaces over successive days. How-

FIGURE 6 Visualization of the neural local volatility surface calibrated with and without the residual of Dupire's equation included as a regularizer.



(a) This row shows the neural local volatility surface calibrated with $\lambda_{\text{dup}} = 1.0$, ie, the residual of Dupire's equation is included as a regularizer. (b) This row shows the neural local volatility surface calibrated with $\lambda_{\text{dup}} = 0.0$, ie, the residual of Dupire's equation is not included as a regularizer. The results are trained with DAX call option prices listed, from left to right, on August 7–9, 2001.

TABLE 2 Comparison of the reprice RMSEs for DAX call options.

	$\lambda_{\text{dup}} = 1$	Tikhonov	$\lambda_{\text{dup}} = 0$	Price-based NN
August 7, 2001	2.85	5.42	9.44	10.18
August 8, 2001	3.27	5.55	6.16	7.44
August 9, 2001	2.67	4.60	8.93	8.18

The columns headed $\lambda_{\text{dup}} = 1$ and $\lambda_{\text{dup}} = 0$ show our results. The Tikhonov values are taken from Crépey (2002). The price-based neural network values are taken from Chataigner *et al* (2020).

ever, to determine whether the variations we observe in Figure 6(b) are linked to overfitting, we resort to the quantitative analysis below.

Like Chataigner *et al* (2020), we assess the calibration of local volatility by computing the reprice RMSEs, and the results are summarized in Table 2.

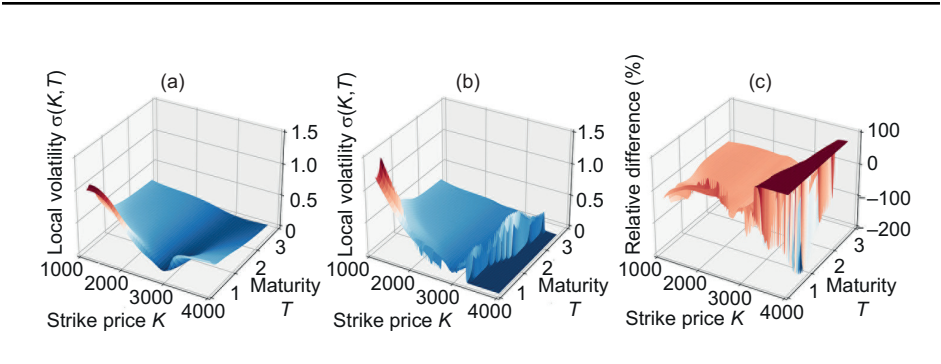
Our numerical results are averaged over three independent runs. The unregularized calibration (ie, $\lambda_{\text{dup}} = 0$) leads to relatively large reprice RMSEs, evidencing that the variations in the local volatility surface over successive days observed in Figure 6 are indeed due to overfitting. On the other hand, compared with previous works using Tikhonov regularization (Crépey 2002) and using a regularizer that imposes positiveness and boundedness of the local volatility (Chataigner *et al* 2020), the inclusion of L_{dup} as a regularizer significantly reduces the reprice RMSEs, leading to the best results. This, therefore, asserts quantitatively the effectiveness of the proposed self-consistent approach.

4.2.2 Put options on the SPX index

In our second application to market data, we use the same data set as Chataigner *et al* (2021); that is, SPX European put options listed on May 18, 2019. The put options are quoted on maturities $T \in [0.055, 2.5]$ and with various strike prices $K \in [1150, 4000]$. The spot price of the underlying is $S_0 = 2859.53$, and $r = 0.023$. Following Chataigner *et al* (2021), the data set is split into a training data set and a testing data set consisting of 1720 and 1725 market put option prices, respectively, enabling a comparison. The calibrated local volatility surfaces for the cases when $\lambda_{\text{dup}} = 0$ and $\lambda_{\text{dup}} = 1$, as well as their relative difference, are shown in Figure 7. We observe that a numerical singularity develops around $K = 3000$ for the case when $\lambda_{\text{dup}} = 0$ is suppressed by the inclusion of L_{dup} as a regularizer.

Next, on the testing data set, we evaluate the interpolation and reprice the RMSEs of the option prices with and without the inclusion of L_{dup} as a regularizer. We then compare our results with published benchmarks obtained using the surface stochastic volatility inspired model, the Gaussian process, and the implied-volatility-based and price-based neural network methods (see Chataigner *et al* (2021) for details). In

FIGURE 7 Visualization of the neural local volatility surface calibrated with and without the inclusion of the residual of Dupire’s equation as a regularizer, and the relative difference of these local volatility surfaces.



(a) Local volatility surface calibrated with the residual of Dupire’s equation as a regularizer (ie, $\lambda_{\text{dup}} = 1$). (b) Local volatility surface calibrated without the residual of Dupire’s equation as a regularizer (ie, $\lambda_{\text{dup}} = 0$). (c) The relative difference of the local volatility surface in part (b) computed with respect to part (a). The results are trained with SPX call option prices listed on May 18, 2019. For ease of visualization, the relative difference in part (c) is truncated at 100%.

TABLE 3 Comparison of interpolation and reprice RMSEs with those of Chataigner *et al* (2021) on SPX put options.

	Chataigner <i>et al</i> (2021)					
	Ours					
	$\lambda_{\text{dup}} = 0$	$\lambda_{\text{dup}} = 1$	SSVI	GP	IV	Price
Interpolation	0.97	1.92	2.89	0.26	2.97	10.35
Reprice	5.24	3.26	22.83	74.02	4.99	11.76

SSVI, surface stochastic volatility inspired model. GP, Gaussian process. IV, implied-volatility-based neural network method. Price, price-based neural network method.

Table 3, the RMSEs are evaluated on the testing grid and our results are averaged over three independent runs.

With $\lambda_{\text{dup}} = 1$, our proposed method achieves the lowest reprice RMSEs on the testing data set, confirming the significance of the proposed self-consistent approach. We note that the Gaussian process, which achieves the lowest interpolation error among all methods, results in the highest reprice RMSEs. This indicates that the market data of the put option prices could contain a significant amount of noise, which may be linked to the singular behavior observed in Figure 7(b), leading ultimately to the high reprice RMSEs in Table 3. Consequently, unlike the case when $\lambda_{\text{dup}} = 0$, a simultaneous increase in interpolation RMSEs and decrease in reprice

RMSEs in the case when $\lambda_{\text{dup}} = 1$ may imply a physics-informed correction has been made to the market data via self-consistency.

5 DISCUSSION AND CONCLUSIONS

In this paper, we introduced a deep learning method that yields the parameterized option price and the calibrated local volatility in a self-consistent manner. More specifically, we approximated both the option price and the local volatility using deep neural networks. Self-consistency was established through Dupire's equation in the sense that the parameterized option price from the market data was required to be a solution to the underlying Dupire equation with calibrated local volatility. Consequently, by exploiting self-consistency, we were able not only to calibrate the local volatility surface from the market option prices but also to filter out noise in the data that violates Dupire's equation with calibrated local volatility, going beyond classical inverse problems with regularization.

The proposed method was tested on both synthetic and market option prices. In all cases, the proposed self-consistent method resulted in a smooth surface for the calibrated local volatility, with smaller reprice RMSEs than those obtained by either canonical methods or deep learning approaches (Achdou and Pironneau 2005; Chataigner *et al* 2020, 2021; Crépey 2002). Moreover, the reprice RMSEs are relatively insensitive to the regularization parameter $\lambda_{\text{dup}} > 0$, demonstrating the robustness of our algorithm. Being continuous functions, the neural networks provide full surfaces for the parameterized option price and the calibrated local volatility, unlike the discrete nodes obtained using canonical methods. However, incorporating the residual of Dupire's equation as a regularizer requires us to solve a two-dimensional partial differential equation at each iteration, leading to increased computation time. This drawback, however, can be mitigated by distributing the training task over multiple graphics cards.

Although option prices may vary dramatically due to the price dynamics of the underlying assets, we observed from Figure 6 that the variation in the local volatility surface over successive days remained small. Therefore, instead of starting from random initial parameters, initiating the training from the converged solutions of the previous day could lead to a substantial reduction in computation time (see Wang and Guet (2022a) for a case study).

DECLARATION OF INTEREST

The authors report no conflicts of interest. The authors alone are responsible for the content and writing of the paper.

ACKNOWLEDGEMENTS

This research was partly carried out under the DesCartes program supported by the National Research Foundation, Prime Minister's Office, Singapore, under its Campus for Research Excellence and Technological Enterprise (CREATE) programme. Zhe Wang is grateful for support from the Energy Research Institute @ NTU, where part of the research underlying this paper was performed.

REFERENCES

- Achdou, Y., and Pironneau, O. (2005). *Computational Methods for Option Pricing*. SIAM, Philadelphia, PA (<https://doi.org/10.1137/1.9780898717495>).
- Ackerer, D., Tagasovska, N., and Vatter, T. (2020). Deep smoothing of the implied volatility surface. In *Proceedings of the 34th Conference on Neural Information Processing Systems*, pp. 11552–11563. Advances in Neural Information Processing Systems, Volume 33. Curran Associates, Red Hook, NY (<https://doi.org/10.2139/ssrn.3402942>).
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research* **18**(153), 1–43. URL: <https://jmlr.org/papers/volume18/17-468/17-468.pdf>.
- Bennett, C. (2014). *Trading Volatility, Correlation, Term Structure and Skew*. CreateSpace. URL: www.trading-volatility.com/Trading-Volatility.pdf.
- Bergomi, L. (2016). *Stochastic Volatility Modeling*. Financial Mathematics Series. Chapman & Hall/CRC, Boca Raton, FL (<https://doi.org/10.1201/b19649>).
- Chataigner, M., Crépey, S., and Dixon, M. (2020). Deep local volatility. *Risks* **8**(3), Paper 82 (<https://doi.org/10.3390/risks8030082>).
- Chataigner, M., Cousin, A., Crépey, S., Dixon, M., and Gueye, D. (2021). Beyond surrogate modeling: learning the local volatility via shape constraints. *SIAM Journal on Financial Mathematics* **12**(3), SC58–SC69 (<https://doi.org/10.1137/20M1381538>).
- Crépey, S. (2002). Calibration of the local volatility in a trinomial tree using Tikhonov regularization. *Inverse Problems* **19**(1), Paper 91 (<https://doi.org/10.1088/0266-5611/19/1/306>).
- Derman, E., and Kani, I. (1994). Riding on a smile. *Risk* **7**(2), 139–145.
- Dupire, B. (1994). Pricing with a smile. *Risk* **7**(1), 18–20. URL: www.risk.net/1530409/.
- Gorban, A. N., and Wunsch, D. C. (1998). The general approximation theorem. In *1998 IEEE International Joint Conference on Neural Networks Proceedings: IEEE World Congress on Computational Intelligence*, Volume 2, pp. 1271–1274. IEEE Press, Piscataway, NJ (<https://doi.org/10.1109/IJCNN.1998.685957>).
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778. IEEE Press, Piscataway, NJ (<https://doi.org/10.1109/CVPR.2016.90>).
- Hull, J. C. (2003). *Options, Futures, and Other Derivatives*, 5th edn. Prentice Hall, Upper Saddle River, NJ.
- Ioffe, S., and Szegedy, C. (2015). Batch normalization: accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pp. 448–456. Proceedings of Machine Learning Research, Volume 37. URL: <https://proceedings.mlr.press/v37/ioffe15.html>.

- Itkin, A. (2020). *Fitting Local Volatility*. World Scientific (<https://doi.org/10.1142/11623>).
- Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., and Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics* **3**, 422–440 (<https://doi.org/10.1038/s42254-021-00314-5>).
- Lin, H., and Jegelka, S. (2018). ResNet with one-neuron hidden layers is a universal approximator. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, pp. 6172–6181. Advances in Neural Information Processing Systems, Volume 31. Curran Associates, Red Hook, NY. URL: <https://dl.acm.org/doi/pdf/10.5555/3327345.3327515>.
- Privault, N. (2002). *Introduction to Stochastic Finance with Market Examples*, 2nd edn. Financial Mathematics Series. Chapman & Hall/CRC, Boca Raton, FL (<https://doi.org/10.1201/9781003298670>).
- Rackauckas, C., Ma, Y., Martensen, J., Warner, C., Zubov, K., Supekar, R., Skinner, D., Ramadhan, A., and Edelman, A. (2020). Universal differential equations for scientific machine learning. Preprint, arXiv (<https://doi.org/10.48550/arXiv.2001.04385>).
- Raissi, M., Perdikaris, P., and Karniadakis, G. E. (2019). Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics* **378**, 686–707 (<https://doi.org/10.1016/j.jcp.2018.10.045>).
- Wang, Z., and Guet, C. (2022a). Deep learning in physics: a study of dielectric quasi-cubic particles in a uniform electric field. *IEEE Transactions on Emerging Topics in Computational Intelligence* **6**(3), 429–438 (<https://doi.org/10.1109/TETCI.2021.3086237>).
- Wang, Z., and Guet, C. (2022b). Self-consistent learning of neural dynamical systems from noisy time series. *IEEE Transactions on Emerging Topics in Computational Intelligence* **6**(5), 1103–1112 (<https://doi.org/10.1109/TETCI.2022.3146332>).
- Winkler, D. A., and Le, T. C. (2017). Performance of deep and shallow neural networks, the universal approximation theorem, activity cliffs, and QSAR. *Molecular Informatics* **36**(1–2), Paper 1600118 (<https://doi.org/10.1002/minf.201600118>).